

# palimpsest

## *User manual*

### Contents

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 1. What palimpsest is for .....                                                   | 3  |
| 2. Installation and compiling .....                                               | 3  |
| 3. Your first revision round .....                                                | 3  |
| 3.1. Recommended workflow .....                                                   | 5  |
| 4. Marking changes: add, del, rep, suppress .....                                 | 5  |
| 4.1. suppress .....                                                               | 6  |
| 5. Anchors: reviewer, editor, author .....                                        | 7  |
| 5.1. Multiple anchors on one passage .....                                        | 7  |
| 5.2. Bare anchors .....                                                           | 8  |
| 5.3. require-exchange .....                                                       | 8  |
| 6. Styling marks: set-revisions .....                                             | 9  |
| 6.1. style .....                                                                  | 9  |
| 6.2. color .....                                                                  | 9  |
| 6.3. add-style / del-style .....                                                  | 10 |
| 6.4. highlight-passage .....                                                      | 11 |
| 6.5. show-anchor .....                                                            | 11 |
| 6.6. del-numbering .....                                                          | 12 |
| 7. Shortcuts: added, deleted, replaced, touched, suppressed .....                 | 16 |
| 7.1. touched .....                                                                | 17 |
| 7.2. suppressed .....                                                             | 17 |
| 8. Writing the exchanges: reviewer, editor, exchange .....                        | 18 |
| 8.1. Co-authors: author, note .....                                               | 19 |
| 8.2. xcomment .....                                                               | 20 |
| 8.3. Header wording: comment-word, change-word, term: .....                       | 20 |
| 9. pinpoint: the manuscript/letter link .....                                     | 21 |
| 9.1. Page only (the default) .....                                                | 21 |
| 9.2. parens: and verb:: fitting the citation into a sentence .....                | 23 |
| 9.3. excerpt: true: quoting the real text .....                                   | 24 |
| 9.4. show-page: and quotes:: dropping the page, adding real quotation marks ..... | 24 |
| 9.5. mode:: overriding style for one excerpt .....                                | 25 |
| 9.6. on-empty: and format: .....                                                  | 27 |
| 10. xref: pointing into the manuscript .....                                      | 27 |
| 11. change-list: a summary table of every marked passage .....                    | 28 |
| 11.1. level:: which heading counts as “Section” .....                             | 29 |
| 11.2. title:: avoiding a redundant heading .....                                  | 30 |
| 12. Working with tables .....                                                     | 31 |
| 12.1. Adding a row or column .....                                                | 31 |
| 12.2. Removing a row or column .....                                              | 32 |
| 12.3. Combining both in one table .....                                           | 33 |
| 13. Bibliography .....                                                            | 34 |
| 13.1. A citation inside add/del .....                                             | 34 |
| 13.2. letter-bibliography: a citation the letter makes on its own .....           | 35 |
| 13.3. Letter numbering .....                                                      | 36 |
| 14. Diagnostics and strict mode .....                                             | 38 |

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 15. Wiring a real project .....                                                     | 39 |
| 15.1. The letter, automatically — matched to the manuscript you actually send ..... | 42 |
| 16. Palimpsest in the contexture ecosystem .....                                    | 44 |
| 16.1. What contexture does .....                                                    | 44 |
| 16.2. @preview/equator: reporting-guideline checklists .....                        | 44 |
| 16.3. Combining the two .....                                                       | 44 |
| 16.4. Where to go next .....                                                        | 48 |

## 1. What palimpsest is for

Peer review means marking every requested change directly in the manuscript, then writing a letter that tells each reviewer exactly where their comment was addressed — quoting the revised wording, citing the exact page. Done by hand, across two or three rounds, that page number is wrong the moment a paragraph moves, and “where did we address R2’s comment 3” means re-reading the whole diff.

Palimpsest automates it. Mark each change where it happens — `passage(<r2-3>)[...add[...]]`, tied to a reviewer’s comment — and one compile produces:

- `manuscript.pdf` — the clean, final version, submittable as-is: every accepted change reads as if it had always been there;
- `manuscript-tracked.pdf` — the same manuscript with every addition underlined and every deletion struck through, colored by whoever asked for it;
- `response.pdf` — a letter, written once, that cites the manuscript’s real page for every response: `pinpoint(<r2-3>)` becomes “p. 4,” correct after every recompile, because the page number comes from the very same compile that just laid the manuscript out.

This only works because all of it comes out of a single Typst compile that can see its own final layout. Palimpsest doesn’t implement that part itself: it’s built on `contexture`, a small shared package installed alongside it (see the next chapter). You don’t need to learn `contexture` to use palimpsest — everything you need is shown here, one function at a time. The closing chapter, “Palimpsest in the contexture ecosystem,” explains what `contexture` actually does and introduces `@preview/equator`, a sibling package for reporting-guideline checklists, for anyone who wants the bigger picture or needs to combine the two.

## 2. Installation and compiling

```
#import "@preview/palimpsest:0.1.0": *
#import "@preview/contexture:0.1.0": bundle
```

A full project (manuscript, tracked manuscript, response letter) compiles with two commands:

```
typst compile --features bundle --format bundle main.typ
typst compile --features bundle --format bundle --input variant=tracked main.typ
```

The first produces `manuscript.pdf` and, once there are responses to answer, `response.pdf`; the second produces `manuscript-tracked.pdf` and, under the same condition, `response-tracked.pdf`. Marking functions — `add`, `del`, `rep`, `passage`, `set-revisions` — also work directly in a single ordinary file, with a plain `typst compile` / `typst compile --input variant=tracked`, no bundle involved — that’s the form used throughout the next few chapters.

Palimpsest itself never calls Typst’s own `document(...)` — that job belongs to `contexture`, the shared engine imported alongside it above. Nothing about that needs to be understood to use the functions in the chapters below; it only becomes relevant once a real, multi-file project gets wired together, in Wiring a real project further down.

Every example in this manual is a single, self-contained file, compiled directly — the simplest way to show what each function does on its own. A real project instead keeps the manuscript, the exchanges, and the wiring in their own files; that shape is introduced once you’ve met the pieces it’s built from.

## 3. Your first revision round

The smallest complete example: one reviewer, two comments, one accepted change and one declined — the whole manuscript/letter pair produced from a single file.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

#let compact(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
```

```

set text(size: 10.5pt)
body
}

#let compact-letter(body) = compact(default-letter-template(body))

#show: contexture.bundle.with(
  template: compact,
  documents: (
    letter(
      exchanges: [
        #reviewer(1)[
          #exchange(<r1-1>)[Please expand the rationale for this design.][
            Done --- #pinpoint(<r1-1>).
          ]
          #exchange(<r1-2>)[This point seems underdeveloped.][
            We believe the current wording is sufficient. #pinpoint(<r1-2>).
          ]
        ]
      ],
      template: compact-letter,
    ),
  ),
)

= Introduction

#added(<r1-1>)[The study rationale, expanded per the reviewer's request.]

#touched(<r1-2>)[This paragraph is unchanged; we address the concern in our response instead.]

```

A few things to notice, top to bottom:

1. `letter(exchanges: [...], template: ...)` describes the response letter; `#show: contexture.bundle.with(documents: (letter(...)),)` is the one line of setup a real project needs — everything else is the manuscript and the exchanges, written exactly as prose.
2. `<r1-1>` and `<r1-2>` are **anchors** — a reviewer number and a comment number, packed into a label. `#exchange(<r1-1>)[comment][response]` writes reviewer 1’s first comment and the reply to it; `#added(<r1-1>)[...]` in the manuscript ties the actual change to that same comment. The full anchor syntax — editor comments, co-author ids, anchors with no response document at all — gets its own chapter next.
3. `#pinpoint(<r1-1>)`, inside the response, is what turns into “p. 1” below — the real page the change landed on, in this very compile.
4. `#touched(<r1-2>)[...]` marks a passage as reviewed but unchanged — useful whenever the honest answer to a comment is “we didn’t change the text, see our response instead,” exactly what the second exchange says.

Compiled plain, `manuscript.pdf` reads like ordinary prose, with no visible trace of any anchor or reviewer:

## Introduction

The study rationale, expanded per the reviewer’s request.

This paragraph is unchanged; we address the concern in our response instead.

Compiled again with `--input variant=tracked`, the accepted change is underlined and both passages carry a small `[R1-1]/[R1-2]` tag, colored by reviewer:

## Introduction

The study rationale, expanded per the reviewer’s request. <sup>[R1-1]</sup>

This paragraph is unchanged; we address the concern in our response instead. <sup>[R1-2]</sup>

And `response.pdf`, from the very same, plain compile — the actual letter, citing real pages:

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*Please expand the rationale for this design.*

Done — (modified on p. 1).

#### Reviewer 1 — comment 2

*This point seems underdeveloped.*

We believe the current wording is sufficient. (see p. 1).

response-tracked.pdf comes out of the second compile the same way, automatically — more on exactly when and why a letter appears at all in The letter, automatically, further down.

### 3.1. Recommended workflow

1. Mark each change in the manuscript as it's made — `#passage(<r1-2>)[...]` wrapping `#add/#del/#rep`, or `#touched(<r1-2>)[...]` where nothing changed but a comment still needs an anchor to point to.
2. Write the exchanges — `#reviewer(n)[...]`, `#exchange(anchor)[comment][response]` — quoting each comment and citing your response with `#pinpoint(anchor)`.
3. Compile twice: plain, and with `--input variant=tracked`. Check that `manuscript.pdf` reads as the real, final submission, and that `manuscript-tracked.pdf` shows every change a reviewer would expect to see.
4. Read `response.pdf/response-tracked.pdf` for diagnostics — a numbered anchor with no matching exchange, an exchange answering nothing, an empty passage. Diagnostics and strict mode covers every case.
5. Before submission, compile once with `strict: true` — every remaining diagnostic becomes a hard compile error instead of a soft marker, a clean pass/fail gate.
6. Submit `manuscript.pdf` (and `manuscript-tracked.pdf`, if the journal wants it) alongside `response.pdf` — all of it came from the same compile(s), so every page the letter cites is guaranteed to match.

### 4. Marking changes: add, del, rep, suppress

`passage(anchors, body)` the citable unit — wraps `body` (plain text and/or `add/del/rep/suppress`) and attaches the anchor(s) a response letter will resolve back to this location. Called as `passage(body)`, with no anchors, for a typo fix with no comment to answer. `anchors` — `<r1-2>`, `<e1>`, a co-author id like `<bob-3>`, or an array of these — gets a full chapter of its own next; every example below either omits it or uses it exactly as seen in the quickstart above.

`add(body)` marks `body` as newly added.

`del(body)` marks `body` as removed.

`rep(old, new)` a replacement in one call.

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#passage[
  Propensity scores were estimated by logistic regression
  #add[and their overlap was assessed graphically].
]

#passage[
  The positivity assumption #rep[was not discussed][is now assessed graphically].
]
```

Compiled once clean:

Propensity scores were estimated by logistic regression and their overlap was assessed graphically.  
The positivity assumption is now assessed graphically.

Once more with `--input variant=tracked`:

Propensity scores were estimated by logistic regression and their overlap was assessed graphically.  
The positivity assumption ~~was not discussed~~ is now assessed graphically.

Two things to notice: `add` shows nothing extra in clean mode — `body` is just there, as if it had always been. `del` shows **nothing at all** in clean mode — no `hide()`, no reserved space; a deletion that survives into the clean version isn't a deletion.

#### 4.1. `suppress`

`suppress(note)` like `del`, but never shows the real content, in **either** mode — only `note`, centered and italic, and only in tracked mode.

```
#import "../lib.typ": *  
  
#set page(width: 16.6cm, height: auto, margin: 12pt)  
#set text(size: 10.5pt)  
  
#passage[  
  The method section stands as before.  
  #suppress[An accompanying diagnostic plot has been removed: it duplicated Figure 2.]  
  The next paragraph is unaffected.  
]
```

The method section stands as before.

*[An accompanying diagnostic plot has been removed: it duplicated Figure 2.]*

The next paragraph is unaffected.

The obvious question is why this exists next to `del`, which already hides its content in clean mode. The difference shows up specifically in the **tracked** manuscript: `del` still renders the real removed content there — struck through, as the previous section demonstrated. Sometimes that's not what you want: a long passage struck through end to end reads worse than a short note saying what used to be there, or the removed content simply isn't worth showing again once it's gone. `suppress` replaces it with `note` instead — centered, italic, shown only in tracked mode — rather than the real thing, struck.

`suppress` takes no content to hide, only `note` — there's no parameter for the real content at all, not even an unused one. Since that content is never rendered by this function, in any mode, keeping it as an argument would only be a place for some future edit to start rendering it by accident, undoing the whole point.

If you want the removed material kept at hand in the source to reconsider later, comment it out beside the call rather than deleting it outright — a `//` line is never evaluated by Typst, so there's no risk of it resurfacing on its own:

```
// #figure(table(...), caption: [The old table.])  
#passage(<r2-1>)[#suppress[Table removed: see response.]]
```

The look of every mark above — color, underline vs. box, whether a small anchor tag shows at the end — is controlled by two things, covered in the next two chapters: the anchor itself (whose shape decides the automatic

per-reviewer/per-author color) and `set-revisions` (any visual knob, changed anywhere in the document). Anchors come first, since a couple of `set-revisions`'s own examples already need one.

## 5. Anchors: reviewer, editor, author

The quickstart above used `<r1-1>/<r1-2>` without explaining the shape. An anchor is parsed into one of three kinds, purely from its shape: `<r1-2>` is reviewer 1, comment 2; `<e1>` is editor comment 1; anything else — `<bob-3>`, `<bob>` — is a co-author id, “bob”, optionally followed by a change number. Each kind gets its own color automatically, without any setup:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(
  require-exchange: false,
  authors: (
    bob: (name: "Bobby Fischer", color: rgb("#c026d3")),
    alice: "Alice Smith",
  ),
)

#added(<r1-1>)[A reviewer's comment, `<r1-1>` --- one color per reviewer number.]

#added(<e1>)[An editor's comment, `<e1>` --- its own fixed color, distinct from every reviewer.]

#added(<bob-1>)[Bob's own change, `<bob-1>` --- name and color both registered explicitly.]

#added(<alice-1>)[Alice's own change, `<alice-1>` --- name registered, color assigned automatically.]

#added(<carol-1>)[Carol's own change, `<carol-1>` --- nothing registered at all, still a distinct color, displayed under her raw id.]
```

[A reviewer's comment, <r1-1> — one color per reviewer number.](#) <sup>[R1-1]</sup>

[An editor's comment, <e1> — its own fixed color, distinct from every reviewer.](#) <sup>[E1]</sup>

[Bob's own change, <bob-1> — name and color both registered explicitly.](#) <sup>[BOB-1]</sup>

[Alice's own change, <alice-1> — name registered, color assigned automatically.](#) <sup>[ALICE-1]</sup>

[Carol's own change, <carol-1> — nothing registered at all, still a distinct color, displayed under her raw id.](#) <sup>[CAROL-1]</sup>

Reviewers are colored by number, cycling through a fixed palette. The editor gets one fixed color of its own. Co-authors draw from a **separate** palette, so a reviewer and a co-author active in the same manuscript are never confusable — and an author's color holds even with nothing configured for them at all (Carol, above): it's a deterministic function of the id text itself, the same color everywhere that id appears. `set-revisions(authors: (...))`, used above for Bob and Alice, additionally gives an id a full display name and/or a specific color:

```
#set-revisions(authors: (
  bob: (name: "Bobby Fischer", color: rgb("#c026d3")), // both
  alice: "Alice Smith",                               // name only
))
```

Either key can be omitted; an id with a `name:` but no `color:` still gets its automatic color, exactly like an id with nothing registered at all.

### 5.1. Multiple anchors on one passage

A comment raised jointly, or addressed in the same place — `passage/added/deleted/replaced` all accept an array of anchors instead of one:

```
#import "../../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

#added(<r1-2>, <r2-1>)[A concern raised jointly by two reviewers, addressed in one place.]
```

[A concern raised jointly by two reviewers, addressed in one place.](#) <sup>[R1-2, R2-1]</sup>

Colored by the first anchor in the list.

## 5.2. Bare anchors

<bob>, with no trailing -<n>, is deliberately **not** meant to key into one particular exchange — it’s the shape for a project with no response document at all, where an anchor just says whose change this is:

```
#import "../../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#added(<dave>)[Dave's first change --- a bare anchor, no number.]

#added(<dave>)[Dave's second change, same bare anchor reused --- no "duplicate" warning, no "no matching exchange" warning either, even though no response document exists anywhere in this example.]
```

[Dave’s first change — a bare anchor, no number.](#) <sup>[DAVE]</sup>

[Dave’s second change, same bare anchor reused — no “duplicate” warning, no “no matching exchange” warning either, even though no response document exists anywhere in this example.](#) <sup>[DAVE]</sup>

Reused as many times as needed, with neither of the two diagnostics that would otherwise apply: no “duplicate exchange” (there’s nothing to be a duplicate **of** — a bare anchor isn’t meant to be unique), and no “no matching exchange” (covered next).

## 5.3. require-exchange

A **numbered** anchor is assumed to answer one specific comment, so passage/added/deleted/replaced check that a matching exchange/note exists somewhere and warn if not:

```
#import "../../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#added(<erin-1>)[A numbered anchor with no matching exchange anywhere --- warns by default.]

#set-revisions(require-exchange: false)

#added(<frank-1>)[Same situation, but `require-exchange: false` is now active --- no warning.]
```

 **anchor erin-1: no matching exchange** [A numbered anchor with no matching exchange anywhere — warns by default.](#) <sup>[ERIN-1]</sup>

[Same situation, but require-exchange: false is now active — no warning.](#) <sup>[FRANK-1]</sup>

`set-revisions(require-exchange: false)` turns this check off from that point on — for a project that wants numbered, colored anchors without ever writing a response document.

## 6. Styling marks: `set-revisions`

Call `set-revisions` once, before any passage, to change how tracked marks look. Takes effect for everything **after** the call — a document can use different styles in different sections.

```
#set-revisions(  
  style: "inline",           // "inline" | "bar" | "none"  
  color: auto,              // auto, or a fixed color for every mark  
  add-style: underline,     // body -> content  
  del-style: (smart default), // body -> content  
  highlight-passage: false, // tint the whole passage's background  
  show-anchor: true,        // show [R1-2] at the end of the passage  
  del-numbering: "none",    // "none" | "keep"  
)
```

### 6.1. style

```
#import "../lib.typ": *  
  
#set page(width: 16.6cm, height: auto, margin: 12pt)  
#set text(size: 10.5pt)  
  
*style: "inline" (default)*  
#passage[This whole sentence is marked #add[with an inline addition] in the flow of text.]  
  
#v(0.8em)  
#set-revisions(style: "bar")  
*style: "bar"*  
#passage[This whole sentence is marked #add[with an addition] under the bar style instead.]  
  
#v(0.8em)  
#set-revisions(style: "none")  
*style: "none"*  
#passage[This whole sentence is marked #add[with an addition] but renders exactly like the clean  
version — a layout sanity check.]
```

**style: “inline” (default)** This whole sentence is marked with an inline addition in the flow of text.

**style: “bar”**

¶ This whole sentence is marked with an addition under the bar style instead.

**style: “none”** This whole sentence is marked with an addition but renders exactly like the clean version — a layout sanity check.

"inline" (the default) underlines additions and strikes deletions in the flow of text. "bar" keeps that same underline/strike on the marks themselves and adds a colored vertical bar to the left of the whole passage's block — best when a passage forms its own block, not text mid-paragraph. "none" turns underline/strike off entirely, rendering exactly like the clean version even in the tracked compile — a layout sanity check, to confirm marking hasn't shifted anything.

### 6.2. color

```
#import "../lib.typ": *  
  
#set page(width: 16.6cm, height: auto, margin: 12pt)  
#set text(size: 10.5pt)  
#set-revisions(require-exchange: false)  
  
*color: auto (default) — one color per reviewer*
```

```
#passage(<r1-1>)[Reviewer 1's #add[addition].]
#passage(<r2-1>)[Reviewer 2's #add[addition] — a different color, automatically.]

#v(0.8em)
#set-revisions(color: rgb("#7a7a7a"))
*color: a fixed value — overrides per-reviewer colors*
#passage(<r1-1>)[Reviewer 1's #add[addition], now in the fixed color.]
#passage(<r2-1>)[Reviewer 2's #add[addition], the same fixed color, not a different one.]
```

**color: auto (default) — one color per reviewer** Reviewer 1's addition.<sup>[R1-1]</sup> Reviewer 2's addition — a different color, automatically.<sup>[R2-1]</sup>

**color: a fixed value — overrides per-reviewer colors** Reviewer 1's addition, now in the fixed color.<sup>[R1-1]</sup> Reviewer 2's addition, the same fixed color, not a different one.<sup>[R2-1]</sup>

auto (the default) colors each mark by its anchor — one color per reviewer, a separate palette per co-author, as introduced in the previous chapter. A fixed color — `rgb(...)`, a named color — overrides that entirely: every mark gets the same color, regardless of anchor.

### 6.3. add-style / del-style

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

*add-style: underline, del-style: default (strikes plain text like this)*
#passage[Some text with #add[an addition] and #del[a deletion].]

#v(0.8em)
#let boxed-mark(body) = box(stroke: 0.6pt, inset: (x: 2pt), radius: 1.5pt, body)
#set-revisions(add-style: boxed-mark, del-style: boxed-mark)
*add-style/del-style: a custom `body -> content` function*
#passage[Some text with #add[an addition] and #del[a deletion], both boxed instead of underlined/struck. The reviewer color still applies on top --- it's added separately by the passage, not by this function.]
```

**add-style: underline, del-style: default (strikes plain text like this)** Some text with an addition and ~~a deletion~~.

**add-style/del-style: a custom body -> content function** Some text with an addition and a deletion, both boxed instead of underlined/struck. The reviewer color still applies on top — it's added separately by the passage, not by this function.

Both take a single `body -> content` function — **not** `(body, color)`: the color is applied afterward, wrapped around whatever this function returns, so a custom style only needs to decide the visual treatment (box, highlight, ...), never the color itself.

add-style's default is plain underline. del-style's default is smarter, because `strike()` never decorates a real math glyph or a figure's drawing — only literal text: ordinary text still gets struck, but a **block** equation, a figure, or a table gets a diagonal cross instead (figures/tables also keep their caption struck natively, since a caption is real text), and an **inline** equation gets a line straight through it rather than a full cross. A custom function passed here always replaces this default outright for every kind of content, the same way it would replace plain `strike`.

Deletions are also automatically desaturated relative to additions — a muted, darker version of the same reviewer’s color, not merely “the same color with a different decoration”. This matters specifically because the strike/cross/line above is the **only** other signal on math or on a figure’s drawing: without a color difference, an added and a removed equation would otherwise look identical at a glance. This isn’t a `del-style` option — it always applies, on top of whatever `del-style` renders.

## 6.4. highlight-passage

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

*highlight-passage: false (default)*
#passage[Only the #add[actually changed words] are tinted --- the rest of the sentence stays plain black text.]

#v(0.8em)
#set-revisions(highlight-passage: true)
*highlight-passage: true*
#passage[The whole passage's background is lightly tinted, not just #add[the changed words] --- useful when a reviewer asked for a full rewrite.]
```

**highlight-passage: false (default)** Only the actually changed words are tinted — the rest of the sentence stays plain black text.

### highlight-passage: true

The whole passage’s background is lightly tinted, not just the changed words — useful when a reviewer asked for a full rewrite.

`false` (the default) tints only the marks themselves — a passage where one word changed doesn’t light up entirely. `true` tints the whole passage’s background lightly, useful when a reviewer asked for a full rewrite and marking only the changed words would understate how much moved.

## 6.5. show-anchor

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

*show-anchor: true (default)*
#passage(<r1-2>)[The #add[anchor tag] appears in superscript at the end of the passage.]

#v(0.8em)
#set-revisions(show-anchor: false)
*show-anchor: false*
#passage(<r1-2>)[No tag at the end of #add[the passage] anymore --- still colored by reviewer, just no visible label.]
```

**show-anchor: true (default)** The anchor tag appears in superscript at the end of the passage. <sup>[R1-2]</sup>

**show-anchor: false** No tag at the end of the passage anymore — still colored by reviewer, just no visible label.

The [R1-2] superscript at the end of a marked passage — what reviewers use most in practice to find their own comment in the manuscript without cross-referencing the letter. `false` removes the tag; the passage is still colored.

## 6.6. del-numbering

A figure, table, equation, or heading that gets deleted still exists as a real element in the tracked manuscript — and by default would still consume a number, shifting every one of its kind that comes after it out of sync with the clean version. `del-numbering: "none"` (the default) keeps the deleted element's own real number visible, struck through, but resets the count right after it, so the **next** real one of that kind keeps the same number in both versions:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set math.equation(numbering: "(1)")
#set heading(numbering: "1.1.")
#set-revisions(require-exchange: false)

*del-numbering: "none" (default)*

= Methods

#figure(rect(width: 2cm, height: 1cm, fill: luma(230)), caption: [Kept figure.]) <fig-a>
#passage(<x1>)[
  #del[#figure(rect(width: 2cm, height: 1cm, fill: luma(230)), caption: [Removed figure.]) <fig-b>]
]
#figure(rect(width: 2cm, height: 1cm, fill: luma(230)), caption: [Next figure --- same number as in the clean version.]) <fig-c>

#figure(table(columns: 2, [Kept], [table]), caption: [Kept table.]) <tab-a>
#passage(<x2>)[
  #del[#figure(table(columns: 2, [Removed], [table]), caption: [Removed table.]) <tab-b>]
]
#figure(table(columns: 2, [Next], [table]), caption: [Next table --- same number as in the clean version.]) <tab-c>

Kept equation. $ a = b $ <eq-a-kept>
#passage(<x3>)[
  #del[$ E = m c^2 $ <eq-a>]
]
Next equation --- same number as in the clean version. $ c = d $ <eq-c-kept>

== Kept subsection.
#passage(<x7>)[
  #del[
    == Removed subsection.
    Removed content.
  ]
]
== Next subsection --- same number as in the clean version.
```

Tracked:

del-numbering: “none” (default)

1. Methods



Figure 1: Kept figure.



Figure 2: Removed figure.

[X1]

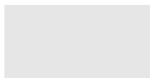


Figure 2: Next figure — same number as in the clean version.

|      |       |
|------|-------|
| Kept | table |
|------|-------|

Table 1: Kept table.

|                    |                  |
|--------------------|------------------|
| <del>Removed</del> | <del>table</del> |
|--------------------|------------------|

Table 2: Removed table.

[X2]

|      |       |
|------|-------|
| Next | table |
|------|-------|

Table 2: Next table — same number as in the clean version.

Kept equation.

$$a = b$$

(1)

(2)

[X3] Next equation — same number as in the clean version.

$$c = d$$

(2)

1.1. Kept subsection.

~~1.2. Removed subsection.~~

~~Removed content.~~ [X7]

1.2. Next subsection — same number as in the clean version.

The **clean** compile of that same source, for comparison — the numbers match:

**del-numbering: "none" (default)**

## 1. Methods



Figure 1: Kept figure.

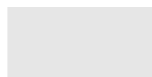


Figure 2: Next figure — same number as in the clean version.

|      |       |
|------|-------|
| Kept | table |
|------|-------|

Table 1: Kept table.

|      |       |
|------|-------|
| Next | table |
|------|-------|

Table 2: Next table — same number as in the clean version.

Kept equation.

$$a = b \tag{1}$$

Next equation — same number as in the clean version.

$$c = d \tag{2}$$

### 1.1. Kept subsection.

### 1.2. Next subsection — same number as in the clean version.

"keep" lets a deleted element consume a number like anything else — rarely what you want (the whole point of `del-numbering: "none"` above is that a reviewer reading the tracked manuscript and citing “Figure 3” is citing the same Figure 3 that exists in the clean version you submit), but available:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set math.equation(numbering: "(1)")
#set heading(numbering: "1.1.")
#set-revisions(require-exchange: false, del-numbering: "keep")

*del-numbering: "keep"*

= Methods

#figure(rect(width: 2cm, height: 1cm, fill: luma(230)), caption: [Kept figure.]) <fig-d>
#passage(<x4>[
  #del[#figure(rect(width: 2cm, height: 1cm, fill: luma(230)), caption: [Removed figure --- still
consumes a number here.]) <fig-e>]
]
#figure(rect(width: 2cm, height: 1cm, fill: luma(230)), caption: [Next figure --- number has
shifted, diverging from the clean version.]) <fig-f>

#figure(table(columns: 2, [Kept], [table]), caption: [Kept table.]) <tab-d>
#passage(<x5>[
  #del[#figure(table(columns: 2, [Removed], [table]), caption: [Removed table --- still consumes a
number here.]) <tab-e>]
]
#figure(table(columns: 2, [Next], [table]), caption: [Next table --- number has shifted, diverging
from the clean version.]) <tab-f>

Kept equation. $ a = b $ <eq-a-kept-b>
#passage(<x6>[
```

```

#del[$ E = m c^2 $ <eq-b>]
]
Next equation --- number has shifted, diverging from the clean version. $ c = d $ <eq-c-kept-b>

== Kept subsection.
#passage(<x8>)[
#del[
== Removed subsection --- still consumes a number here.
Removed content.
]
]
== Next subsection --- number has shifted, diverging from the clean version.

```

Tracked:

del-numbering: “keep”

1. Methods



Figure 1: Kept figure.



Figure 2: Removed figure — still consumes a number here.

[X4]

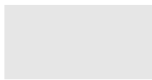


Figure 3: Next figure — number has shifted, diverging from the clean version.

|      |       |
|------|-------|
| Kept | table |
|------|-------|

Table 1: Kept table.

|                    |                  |
|--------------------|------------------|
| <del>Removed</del> | <del>table</del> |
|--------------------|------------------|

Table 2: Removed table — still consumes a number here.

[X5]

|      |       |
|------|-------|
| Next | table |
|------|-------|

Table 3: Next table — number has shifted, diverging from the clean version.

Kept equation.

$$a = b$$
(1)

$$E = mc^2$$
(2)

[X6] Next equation — number has shifted, diverging from the clean version.

$$c = d$$
(3)

1.1. Kept subsection.

1.2. Removed subsection — still consumes a number here.

Removed content. [X8]

1.3. Next subsection — number has shifted, diverging from the clean version.

The **clean** compile of that same source — watch the third and fifth figures' numbers diverge from the tracked version above:

**del-numbering: "keep"**

## 1. Methods

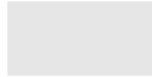


Figure 1: Kept figure.

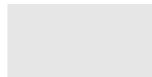


Figure 2: Next figure — number has shifted, diverging from the clean version.

|      |       |
|------|-------|
| Kept | table |
|------|-------|

Table 1: Kept table.

|      |       |
|------|-------|
| Next | table |
|------|-------|

Table 2: Next table — number has shifted, diverging from the clean version.

Kept equation.

$$a = b \tag{1}$$

Next equation — number has shifted, diverging from the clean version.

$$c = d \tag{2}$$

### 1.1. Kept subsection.

### 1.2. Next subsection — number has shifted, diverging from the clean version.

Both examples also include a table, an equation, and a heading, deleted the same way, worth looking at closely: the table's cells and caption are struck through, same as ordinary text, and its diagonal cross covers the drawing itself. The equation gets the same diagonal cross rather than a strikethrough (see `del-style` above for why), and the heading is struck like any other text. All three keep their own number visible, frozen under `del-numbering: "none"` so whatever comes after them stays in sync with the clean version, or consume a real number under `"keep"`, exactly the same freeze/consume distinction shown above for figures.

This works the same way under a template that recomputes its own figure or table numbers via a custom show rule — `@preview/charged-ieee`, for instance — since the underlying counter it reads is the same one being frozen here.

## 7. Shortcuts: added, deleted, replaced, touched, suppressed

`add/del/rep` mark part of a passage — a clause added mid-sentence, one phrase replacing another. When the **entire** passage is new, removed, or rewritten, wrapping it by hand (`passage(anchor) [#add[...]]`) is one level of nesting that never varies — these five shortcuts skip it.

`added(anchors, body, summary: none) passage(anchors, add(body)).`

`deleted(anchors, body, summary: none) passage(anchors, del(body)).`

`replaced(anchors, old, new, summary: none) passage(anchors, rep(old, new)).`

```
#import "../lib.typ": *
#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
```

```
#set-revisions(require-exchange: false)

#added(<r1-1>)[This whole sentence is new.]

#deleted(<r1-2>)[This whole sentence is being removed.]

#replaced(<r1-3>)[The old wording.][The new wording.]
```

This whole sentence is new. <sup>[R1-1]</sup>

~~This whole sentence is being removed.~~ <sup>[R1-2]</sup>

~~The old wording.~~ The new wording. <sup>[R1-3]</sup>

summary: is accepted by all three but only matters once a passage’s tracked appearance gives a letter nothing worth quoting — deleted is the common case, covered below under suppressed.

## 7.1. touched

touched(anchors, body) declares a location without marking any change — “we checked this, it stays as written.” Unlike passage on its own, an anchor with no add/del/rep inside doesn’t need allow-empty: to avoid a warning; touched already sets it.

```
#import "../..lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

#touched(<r1-4>)[This paragraph has not changed; we address the reviewer's concern in our response instead.]
```

This paragraph has not changed; we address the reviewer’s concern in our response instead. <sup>[R1-4]</sup>

Still colored and tagged like any other passage — just nothing struck or underlined, since nothing changed.

## 7.2. suppressed

suppressed(anchors, note, summary: auto) passage(anchors, suppress(note), summary: ...) — suppress (Marking changes, above) for a passage entirely made of it.

```
#import "../..lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

#suppressed(<r2-1>, [Equation removed: superseded by the equation below.])

$ E = m c^2 (1 + epsilon) $

#suppressed(
  <r2-2>,
  [Table removed: see response to R2-2.], // shown in the tracked manuscript
  summary: [the old sensitivity table], // for the letter, later
)
```

*[Equation removed: superseded by the equation below.]*

[R2-1]

$$E = mc^2(1 + \varepsilon)$$

*[Table removed: see response to R2-2.]*

[R2-2]

`note` and `summary`: are separate parameters on purpose. `note` has to read on its own, cold, in the middle of the manuscript — “Equation removed: ...”. `summary`: is inserted into a sentence the **letter** will already have written — “Removed: `<summary>`” — so it should be a bare phrase, not repeat “removed” itself. `summary`: defaults to `note` when omitted, since most of the time the two don’t need to differ; the second call above gives them separately.

## 8. Writing the exchanges: reviewer, editor, exchange

`reviewer(n, body)` groups a reviewer’s comments under a heading colored by their number.

`editor(body)` same, for the editor’s own comments.

`exchange(anchor, comment, response)` renders the quoted comment (in italics) and the response, with a header generated from the anchor.

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

// Manuscript-side passages, matched by the exchanges below.
#added(<r1-1>)[A justification for the sample size.]
#added(<r1-2>)[A sensitivity analysis, added alongside it.]
#added(<e1>)[A clarified abstract word count.]

// Response-side exchanges -- a reviewer's block holds as many
// successive exchanges as needed, one heading for the whole group.
#reviewer(1)[
  #exchange(<r1-1>)[The sample size is not justified.][
    We agree and have added a justification.
  ]

  #exchange(<r1-2>)[A sensitivity analysis would strengthen this.][
    Added, using the same assumptions.
  ]
]

#editor[
  #exchange(<e1>)[Please double-check the abstract word count.][
    Confirmed within the journal's limit.
  ]
]
```

[A justification for the sample size.](#) <sup>[R1-1]</sup> [A sensitivity analysis, added alongside it.](#) <sup>[R1-2]</sup> [A clarified abstract word count.](#) <sup>[E1]</sup>

## Reviewer 1

### Reviewer 1 — comment 1

*The sample size is not justified.*

We agree and have added a justification.

### Reviewer 1 — comment 2

*A sensitivity analysis would strengthen this.*

Added, using the same assumptions.

## Editor

### Editor — comment 1

*Please double-check the abstract word count.*

Confirmed within the journal's limit.

exchange checks, unconditionally, that its anchor matches a passage somewhere — an orphan comment answering nothing is always worth flagging, so this check has no `require-exchange-style` opt-out.

## 8.1. Co-authors: author, note

`author(id, body)` groups one co-author's own notes under a heading, colored and named the same way an anchor like `<bob-3>` already would be.

`note(anchor, text)` a single block — no comment to quote, just the author's own explanation. `exchange(anchor, text)`, with two arguments instead of three, renders identically.

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(authors: (bob: "Bobby Fischer"))

// Manuscript-side passages.
#added(<bob-1>)[Bobby's own addition.]
#added(<bob-2>)[Bobby's second change.]

// Response-side notes -- no reviewer comment to quote, just an author
// explaining their own change.
#author("bob"){
  #note(<bob-1>)[Explaining why I made this change.]

  // exchange() with two arguments renders identically to note() above.
  #exchange(<bob-2>)[Explaining the second change, via exchange() instead.]
}
```

[Bobby's own addition.](#) <sup>[BOB-1]</sup> [Bobby's second change.](#) <sup>[BOB-2]</sup>

## Bobby Fischer

### Bobby Fischer — change 1

Explaining why I made this change.

### Bobby Fischer — change 2

Explaining the second change, via `exchange()` instead.

## 8.2. xcomment

`xcomment(anchor)` a clickable cross-reference to another exchange, plus its page — “as already answered in comment R1-2.”

```
#import "../..lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#added(<r1-1>)[The scope was narrowed to three use cases.]
#added(<r1-2>)[A sensitivity analysis was added, consistent with the narrower scope.]

#reviewer(1)[
  #exchange(<r1-1>)[The scope seems too broad.][
    Narrowed as suggested.
  ]

  #exchange(<r1-2>)[A sensitivity analysis would help.][
    Added --- see also our response to #xcomment(<r1-1>).
  ]
]
```

[The scope was narrowed to three use cases.](#) <sup>[R1-1]</sup> [A sensitivity analysis was added, consistent with the narrower scope.](#) <sup>[R1-2]</sup>

### Reviewer 1

#### Reviewer 1 — comment 1

*The scope seems too broad.*

Narrowed as suggested.

#### Reviewer 1 — comment 2

*A sensitivity analysis would help.*

Added — see also our response to reviewer 1, comment 1, p. 1.

## 8.3. Header wording: comment-word, change-word, term:

The noun in a header — “comment” for reviewer/editor, “change” for a co-author — is `set-revisions(comment-word:, change-word:)`, global from that point on, or `term: on one exchange/note call for just that occurrence:`

```
#import "../..lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(authors: (bob: "Bobby Fischer"))

#added(<r1-1>)[Default word: "comment".]
#added(<bob-1>)[Default word: "change".]
#added(<r2-1>)[Global override applies here.]
#added(<bob-2>)[Per-call override applies here.]

#reviewer(1)[
  #exchange(<r1-1>)[A remark.][The default reviewer word.]
]
#author("bob")[
  #note(<bob-1>)[The default author word.]
]

#set-revisions(comment-word: "remark", change-word: "revision")

#reviewer(2)[
  #exchange(<r2-1>)[Another remark.][Now under the global override --- "remark", not "comment".]
]

#author("bob")[
  #note(<bob-2>, term: "aside")[This one overrides the word for just this call.]
]
```

```
]

```

Cross-references: `#xcomment(<r1-1>)`, `#xcomment(<r2-1>)`, and `#xcomment(<bob-2>)`.

[Default word: “comment”.](#)<sup>[R1-1]</sup> [Default word: “change”.](#)<sup>[BOB-1]</sup> [Global override applies here.](#)<sup>[R2-1]</sup> [Per-call override applies here.](#)<sup>[BOB-2]</sup>

## Reviewer 1

### Reviewer 1 — comment 1

*A remark.*

The default reviewer word.

## Bobby Fischer

### Bobby Fischer — change 1

The default author word.

## Reviewer 2

### Reviewer 2 — remark 1

*Another remark.*

Now under the global override — “remark”, not “comment”.

## Bobby Fischer

### Bobby Fischer — aside 2

This one overrides the word for just this call.

Cross-references: reviewer 1, remark 1, p. 1, reviewer 2, remark 1, p. 1, and Bobby Fischer’s aside 2, p. 1.

`xcomment` echoes whichever word the exchange it points to actually used — reading it back from that exchange’s own data rather than recomputing it, so `#xcomment(<bob-2>)`’s explicit term: “aside” stays “aside” no matter where it’s cited from. An **unoverridden** word, though, is resolved at the position where it’s **read**, not where it was written: `<r1-1>`’s own exchange, near the top, renders “comment” — the default, in effect at that point — but `#xcomment(<r1-1>)` at the very bottom, after the global override, reads back “remark” for that same exchange. A global change to `comment-word/change-word` partway through a file affects every unoverridden reference read afterward, regardless of which word was showing where that exchange itself was originally written.

## 9. pinpoint: the manuscript/letter link

```
#pinpoint(anchor, excerpt: false, parens: true, verb: auto, show-page: true, quotes: false,
format: auto, mode: auto, on-empty: auto)
```

`pinpoint(<anchor>)` searches the manuscript for every passage carrying that anchor and reports where it is — the mechanism behind the real page numbers a response letter cites, since the manuscript and the letter share one bundle.

### 9.1. Page only (the default)

```
#import "../lib.typ": *

#document("manuscript.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #passage(<r1-1>)[The sample size #rep[was not justified][was determined by a power
calculation].]
]

#document("response.png")[
```

```
#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#reviewer(1)[
  #exchange(<r1-1>)[The sample size is not justified.][
    We thank the reviewer. #pinpoint(<r1-1>)
  ]
]
```

Manuscript:

The sample size was determined by a power calculation.

Response:

## Reviewer 1

### Reviewer 1 — comment 1

*The sample size is not justified.*

We thank the reviewer. (modified on p. 1)

The same anchor on two passages, on different pages, reports both:

```
#import "../lib.typ": *
#document("manuscript.pdf")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #passage(<r1-1>)[This revision responds to a single comment that required changes in two
separate sections.]
  #pagebreak()
  #passage(<r1-1>)[The same comment, addressed a second time here.]
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #reviewer(1)[
    #exchange(<r1-1>)[A single comment requiring changes in two places.][
      Addressed in both places. #pinpoint(<r1-1>)
    ]
  ]
]
```

Manuscript, page 1:

This revision responds to a single comment that required changes in two separate sections.

Manuscript, page 2:

The same comment, addressed a second time here.

Response:

## Reviewer 1

### Reviewer 1 — comment 1

*A single comment requiring changes in two places.*

Addressed in both places. (see p. 1 and p. 2)

If the anchor matches no real add/del/rep mark anywhere — a touched passage, cited only to point at text that didn't change — the verb switches from “modified on” to “see” automatically. Not a style choice: asserting a change that didn't happen would simply be false, so nothing has to be configured for it.

## 9.2. parens: and verb:: fitting the citation into a sentence

The wired-in parenthetical reads fine as a trailing citation (“We addressed this concern (modified on p. 3).”), but not every sentence ends that way:

```
#import "../lib.typ": *

#document("manuscript.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #passage(<r1-1>)[The sample size #rep[was not justified][was determined by a power
calculation].]
  #touched(<r1-2>)[We kept the eligibility criteria exactly as submitted.]
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #reviewer(1)[
    #exchange(<r1-1>)[The sample size is not justified.][
      See #pinpoint(<r1-1>, parens: false, verb: none) for the updated wording.
    ]

    #exchange(<r1-2>)[Please confirm the eligibility criteria are unchanged.][
      Confirmed --- #pinpoint(<r1-2>).
    ]
  ]
]
```

Manuscript:

The sample size was determined by a power calculation. We kept the eligibility criteria exactly as submitted.

Response:

## Reviewer 1

### Reviewer 1 — comment 1

*The sample size is not justified.*

See p. 1 for the updated wording.

### Reviewer 1 — comment 2

*Please confirm the eligibility criteria are unchanged.*

Confirmed — (see p. 1).

parens: false drops the parentheses; verb: none additionally drops “modified on”/“see”, leaving only “p. 3” (or “p. 3 and p. 7” for two pages) — for a sentence, like See #pinpoint(<r>, parens: false, verb: none) for the

updated wording., that already supplies its own verb. The two are independent: `parens: false` alone still says “modified on p. 3” without the parentheses; `verb: none` alone keeps the parentheses around a bare page number. Neither one alone fits every sentence shape — that’s why both exist rather than a single “compact” switch.

### 9.3. `excerpt: true`: quoting the real text

Rather than a page number, the actual content of every passage carrying the anchor — exactly as it reads in the manuscript right now, so it can never go stale:

```
#import "../lib.typ": *

#document("manuscript.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #passage(<r1-1>)[The sample size #rep[was not justified][was determined by a power
calculation].]
  #deleted(<r1-2>, summary: [an outdated caveat about generalizability])[
    An earlier caveat about generalizability, no longer needed.
  ]
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #reviewer(1)[
    #exchange(<r1-1>)[The sample size is not justified.][
      Addressed as follows: #pinpoint(<r1-1>, excerpt: true)
    ]

    #exchange(<r1-2>)[This caveat is no longer accurate.][
      Agreed: #pinpoint(<r1-2>, excerpt: true)
    ]
  ]
]
```

Manuscript:

The sample size was determined by a power calculation.

Response:

#### Reviewer 1

##### Reviewer 1 — comment 1

*The sample size is not justified.*

Addressed as follows: **p. 1** — The sample size was determined by a power calculation.

##### Reviewer 1 — comment 2

*This caveat is no longer accurate.*

Agreed: Removed: an outdated caveat about generalizability.

A passage declared with `summary:` ignores `excerpt` and always renders “Removed: <summary>” instead, as the second comment above shows — there being no meaningful text left to quote in the clean manuscript once the passage is fully removed.

### 9.4. `show-page: and` `quotes::` dropping the page, adding real quotation marks

```
#import "../lib.typ": *

#document("manuscript.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
```

```

#passage(<r1-1>)[The sample size #rep[was not justified][was determined by a power
calculation].]
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #reviewer(1)[
    #exchange(<r1-1>)[The sample size is not justified.][
      #pinpoint(<r1-1>, excerpt: true, quotes: true)

      On page 1: #pinpoint(<r1-1>, excerpt: true, show-page: false)
    ]
  ]
]

```

Manuscript:

The sample size was determined by a power calculation.

Response:

## Reviewer 1

### Reviewer 1 — comment 1

*The sample size is not justified.*

**p. 1** — “The sample size was determined by a power calculation.”

On page 1: The sample size was determined by a power calculation.

`show-page: false` drops the leading `*p. 1* ---` — for a sentence that already states where the excerpt comes from (“On page 1, `<excerpt>`”), or a caller who simply doesn’t want it. `quotes: true` wraps the excerpt in real, typeset quotation marks via Typst’s native `quote()`.

`quotes: true` only ever **requests** quotation marks — a passage whose content is a figure, a table, or a block equation never gets them, regardless of this setting: forcing quotation marks onto a figure produces two stray quote glyphs sitting alone above and below it, not an improvement, so this is detected and declined automatically rather than left for you to remember passage by passage.

## 9.5. `mode:: overriding style for one excerpt`

An excerpt renders, by default, in whichever mode the **current** compile is running under — clean text in `response.pdf`, struck-through/underlined tracked style in `response-tracked.pdf` (both produced automatically once `exchanges` is set, see below). `mode:` overrides this for one call, in either direction.

`mode: "tracked"`, called from a clean letter, shows the tracked style even though `response.pdf` itself is otherwise all clean text:

```

#import "../lib.typ": *

#document("manuscript.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #passage(<r1-1>)[The sample size #rep[was not justified][was determined by a power
calculation].]
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #reviewer(1)[
    #exchange(<r1-1>)[The sample size is not justified.][

```

```
    Compare the wording: #pinpoint(<r1-1>, excerpt: true, mode: "tracked")
  ]
]
```

Manuscript:

The sample size was determined by a power calculation.

Response:

## Reviewer 1

### Reviewer 1 — comment 1

*The sample size is not justified.*

Compare the wording: p. 1 — The sample size ~~was not justified~~ [was determined by a power calculation](#).

Useful when the point being made is “we removed exactly what you objected to,” which a clean, final-text quote doesn’t convey on its own.

The reverse also works: mode: "clean", called from a letter compiled tracked (--input variant=tracked, exchanges set), shows the final wording for one excerpt even though the rest of that same letter is otherwise quoting tracked style:

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

#let my-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  body
}

#let exchanges = reviewer(1)[
  #exchange(<r1-1>)[This claim needs more support.][
    We have expanded this sentence. #pinpoint(<r1-1>, excerpt: true, mode: "clean")
  ]
]

#show: contexture.bundle.with(
  template: my-template,
  documents: (letter(exchanges: exchanges, template: my-template)),
)

#passage(<r1-1>)[The treatment #rep[has an effect][has a clinically meaningful effect] on
survival.]
```

Manuscript, tracked:

The treatment ~~has an effect~~ [has a clinically meaningful effect](#) on survival. <sup>[R1-1]</sup>

Response, tracked — the excerpt still reads as accepted, final text, despite the compile being tracked:

## Reviewer 1

### Reviewer 1 — comment 1

*This claim needs more support.*

We have expanded this sentence. **p. 1** — The treatment has a clinically meaningful effect on survival.

## 9.6. on-empty: **and** format:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

No anchor `<r9-9>` exists anywhere in this document.

Default `on-empty: warns: #pinpoint(<r9-9>)`

#pinpoint(<r9-9>, on-empty: none) shows nothing instead when given `on-empty: none`.

#pinpoint(<r9-9>, on-empty: [not found in this round]) shows custom content instead, given as `on-empty:`.

#passage(<r2-1>)[A change #add[on this page].]

Default wording: #pinpoint(<r2-1>)

Custom `format:: #pinpoint(<r2-1>, format: (pages, has-marks) => [(see line 1 of p.
#pages.first())])`
```

No anchor <r9-9> exists anywhere in this document.

Default on-empty: warns:  **pinpoint(<r9-9>): no revision attached to this anchor**

shows nothing instead when given on-empty: none.

not found in this round shows custom content instead, given as on-empty:.

A change on this page.

Default wording: (modified on p. 1)

Custom format:: (see line 1 of p. 1)

on-empty: controls what happens when no passage anywhere carries the given anchor — auto (the default) warns, none shows nothing, anything else is shown as-is. format: replaces the default page-list wording entirely, with a function (pages-array, has-marks) -> content — has-marks is the same flag that drives the automatic “modified”/“see” switch above, passed through so a fully custom format can make that same distinction without querying the bundle again. For a journal with its own citation convention.

## 10. xref: pointing into the manuscript

xref(label) like @label/ref(label) — already correct across the bundle, resolving to the manuscript’s real number for free — but with the manuscript’s real page number appended.

```
#import "../lib.typ": *

#document("manuscript.pdf")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #set heading(numbering: "1.")
  = Results
```

```

#pagebreak()
#figure(rect(width: 3cm, height: 1.5cm, fill: luma(230)), caption: [Propensity score
distribution.]) <fig-a>
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)

  A bare reference resolves the real manuscript number, for free: @fig-a.

  `xref` adds the page number on top: #xref(<fig-a>).

  A label that doesn't exist: #xref(<fig-nonexistent>).
]

```

Manuscript, page 2:

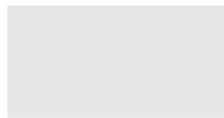


Figure 1: Propensity score distribution.

Response:

A bare reference resolves the real manuscript number, for free: Figure 1.

xref adds the page number on top: Figure 1, p. 2.

A label that doesn't exist: ! xref(<fig-nonexistent>): not found.

A bare @fig-a already gets the number right, since the letter and the manuscript share one bundle — xref only adds the page. A label that doesn't exist anywhere warns rather than breaking the compile.

## 11. change-list: a summary table of every marked passage

change-list(title: [Summary of changes], level: 1) one row per marked passage — comment, type of change, page, section.

```

#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set heading(numbering: "1.")
#set-revisions(require-exchange: false, authors: (bob: "Bobby Fischer"))

#change-list()

= Introduction

#added(<r1-1>)[A reviewer's addition.]
#deleted(<bob-1>)[Bobby's own deletion.]

== Background

#replaced(<r1-2>)[A second reviewer comment, addressed in this subsection.][A second reviewer
comment, revised in this subsection.]

= Methods

#added(<e1>)[An editor's addition.]
#added(none)[An anonymous typo fix, no anchor attached.]
#touched(<r9-9>)[This paragraph is unchanged --- never listed below.]

```

## Summary of changes

| Comment | Type        | Page | Section      |
|---------|-------------|------|--------------|
| R1-1    | addition    | 1    | Introduction |
| R1-2    | replacement | 1    | Introduction |
| BOB-1   | deletion    | 1    | Introduction |
| E1      | addition    | 1    | Methods      |
| —       | addition    | 1    | Methods      |

## 1. Introduction

[A reviewer's addition.](#) <sup>[R1-1]</sup> ~~Bobby's own deletion.~~ <sup>[BOB-1]</sup>

### 1.1. Background

~~A second reviewer comment, addressed in this subsection.~~ [A second reviewer comment, revised in this subsection.](#) <sup>[R1-2]</sup>

## 2. Methods

[An editor's addition.](#) <sup>[E1]</sup> [An anonymous typo fix, no anchor attached.](#) This paragraph is unchanged — never listed below. <sup>[R9-9]</sup>

Renders nothing in clean mode, like `del/suppress` — placed once anywhere in shared manuscript content, it only ever shows up in `manuscript-tracked.pdf`. Rows are sorted by comment identifier — reviewers in order, then co-authors alphabetically by id, then the editor, anchor-less passages last — so the table doubles as a checklist against the letter, not just a readout of document order. `touched` never appears: nothing changed there to list. A passage with several marks shows every kind it contains, joined with “/” (addition/deletion, say, for a manually mixed passage that isn't a single rep).

The “Section” column is the nearest preceding heading at `level`: (top-level by default) — notice the reviewer comment inside “Background” above is listed under “Introduction”, its enclosing top-level section, not the subsection itself.

### 11.1. `level`:: which heading counts as “Section”

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set heading(numbering: "1.")
#set-revisions(require-exchange: false)

*level: 1 (default) --- "Section" names the top-level heading:*

#change-list()

*level: 2 --- "Section" names the subsection instead:*

#change-list(level: 2)

= Introduction

== Background

#added(<r1-1>)[A reviewer's addition, inside a subsection.]
```

**level: 1 (default) — “Section” names the top-level heading:**

**Summary of changes**

| Comment | Type     | Page | Section      |
|---------|----------|------|--------------|
| R1-1    | addition | 1    | Introduction |

**level: 2 — “Section” names the subsection instead:**

**Summary of changes**

| Comment | Type     | Page | Section    |
|---------|----------|------|------------|
| R1-1    | addition | 1    | Background |

## 1. Introduction

### 1.1. Background

[A reviewer’s addition, inside a subsection.](#) <sup>[R1-1]</sup>

Same passage, same position — only `level:` differs between the two calls, and the `Section` column changes with it: `1` (the default) names the enclosing top-level heading, `2` the enclosing subsection.

## 11.2. `title::` avoiding a redundant heading

`change-list()` prints its own bold “Summary of changes” line above the table by default. If you already write your own heading right before the call, that becomes two headings back to back:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

*Without `title: none` --- your own heading, then the package's own "Summary of changes" on top of it:*

#text(weight: "bold", size: 1.2em)[Summary of Revisions]
#change-list()

*With `title: none` --- no second, redundant heading:*

#text(weight: "bold", size: 1.2em)[Summary of Revisions]
#change-list(title: none)

#added(<r1-1>)[A reviewer's addition.]
```

Without `title: none` — your own heading, then the package’s own “Summary of changes” on top of it:

## Summary of Revisions

### Summary of changes

| Comment | Type     | Page | Section |
|---------|----------|------|---------|
| R1-1    | addition | 1    | —       |

With `title: none` — no second, redundant heading:

## Summary of Revisions

| Comment | Type     | Page | Section |
|---------|----------|------|---------|
| R1-1    | addition | 1    | —       |

[A reviewer’s addition.](#) <sup>[R1-1]</sup>

`title: none` removes the package’s own line, leaving just your heading and the table — `title: [Some other text]` would replace it with different wording instead, for the same reason.

## 12. Working with tables

`add/de1` mark **content** — a word, a cell, a whole figure — but a table’s own shape is different: Typst’s `table()` has no built-in way to conditionally include or exclude an entire row or column depending on the compile mode. Getting this wrong doesn’t just look off — it can leave a row that reviewers were told was added missing from the manuscript actually submitted, or a removed row still sitting in it.

### 12.1. Adding a row or column

An added row or column stays in the table in **both** compiles, exactly like any other `#add[...]` — accepted new content never disappears from the clean version. No `mode()` check at all: wrap each cell in `add[...]` and include the row unconditionally, same as writing any other table row by hand.

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#passage[
  #figure(
    table(
      columns: 2,
      [Group], [N],
      [Control], [42],
      add[Treatment], add[45],
    ),
    caption: [Sample sizes by group.],
  )
]
```

Clean:

| Group     | N  |
|-----------|----|
| Control   | 42 |
| Treatment | 45 |

Table 1: Sample sizes by group.

Tracked:

| Group            | N         |
|------------------|-----------|
| Control          | 42        |
| <u>Treatment</u> | <u>45</u> |

Table 1: Sample sizes by group.

## 12.2. Removing a row or column

The opposite: a genuinely removed row or column must be **absent** from the clean compile, and there's no native way to hide part of a table conditionally. The fix is to build that row's cells as a spread array that's empty in clean and populated in tracked — `..if mode() == "clean" { () } else { (del[...], ...) }` — the same `mode()` already used above for `require-exchange-free` examples, exported by `lib.typ` for exactly this.

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#passage[
  #figure(
    table(
      columns: 2,
      [Group], [N],
      [Control], [42],
      ..if mode() == "clean" { () } else { (del[Treatment], del[45]) },
      [Placebo], [40],
    ),
    caption: [Sample sizes by group.],
  )
]
```

Clean:

| Group   | N  |
|---------|----|
| Control | 42 |
| Placebo | 40 |

Table 1: Sample sizes by group.

Tracked:

| Group     | N  |
|-----------|----|
| Control   | 42 |
| Treatment | 45 |
| Placebo   | 40 |

Table 1: Sample sizes by group.

### 12.3. Combining both in one table

A table can have an added row **and** a removed row at the same time — the two patterns above coexist without conflict, since neither changes `columns`:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#passage[
  #figure(
    table(
      columns: 2,
      [Group], [N],
      [Control], [42],
      ..if mode() == "clean" { ( ) } else { (del[Placebo], del[40]) },
      add[Treatment], add[45],
    ),
    caption: [Sample sizes by group --- Placebo dropped, Treatment added.],
  )
]
```

Clean:

| Group     | N  |
|-----------|----|
| Control   | 42 |
| Treatment | 45 |

Table 1: Sample sizes by group — Placebo dropped, Treatment added.

Tracked:

| Group            | N         |
|------------------|-----------|
| Control          | 42        |
| Placebo          | 40        |
| <u>Treatment</u> | <u>45</u> |

Table 1: Sample sizes by group — Placebo dropped, Treatment added.

**Columns are the trap.** `columns`: is one fixed count for the whole table, so an added column and a removed column do **not** belong in the same delta: an added column is part of the fixed base count (present in both compiles, like the row case above), and only a **removed** column belongs in a mode-dependent addition to that count — `base + int(mode() != "clean")` per removed column, never per added one. Writing it the other way around silently drops the added column from the clean, submitted manuscript — easy to miss, since `manuscript-tracked.pdf` still looks right.

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

// Only the removed column enters the mode-dependent delta --- the
// added column is part of the fixed base count, present in both
// compiles, same as the added row above.
#passage[
  #figure(
    table(
      columns: 3 + int(mode() != "clean"),
      [Group], [N], add[p-value], ..if mode() == "clean" { () } else { (del[Old score],) },
      [Control], [42], add[0.03], ..if mode() == "clean" { () } else { (del[--],) },
    ),
    caption: [Sample sizes --- p-value column added, Old score column dropped.],
  )
]
```

Clean:

| Group   | N  | p-value |
|---------|----|---------|
| Control | 42 | 0.03    |

Table 1: Sample sizes — p-value column added, Old score column dropped.

Tracked:

| Group   | N  | <u>p-value</u> | <del>Old score</del> |
|---------|----|----------------|----------------------|
| Control | 42 | <u>0.03</u>    | —                    |

Table 1: Sample sizes — p-value column added, Old score column dropped.

## 13. Bibliography

Typst 0.15's multiple bibliographies handle three situations.

### 13.1. A citation inside `add/del`

Nothing special to do — it becomes part of the manuscript's own bibliography either way. The difference shows up in **which version** the citation appears in:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)
#set-revisions(require-exchange: false)

#added(<r1-1>)[A new claim, supported by @smith2020.]

#deleted(<r1-2>, summary: [an outdated claim])[
  An outdated claim, previously supported by @jones2019.
]

#bibliography("../typst-palimpsest/docs/manual-snippets/biblio-manuscript.bib")
```

Clean — only the surviving citation:

A new claim, supported by [1].

## Bibliography

[1] J. Smith, “A relevant finding,” *Journal of Examples*, 2020.

Tracked — both, since the struck-through text is still there to resolve:

~~A new claim, supported by [1].~~<sup>[R1-1]</sup>

~~An outdated claim, previously supported by [2].~~<sup>[R1-2]</sup>

## Bibliography

[1] J. Smith, “A relevant finding,” *Journal of Examples*, 2020.

[2] A. Jones, “An outdated finding,” *Journal of Examples*, 2019.

A citation inside a deleted passage disappears from the bibliography in the clean compile and reappears in the tracked one, so a reader following the struck-through text can still resolve what it cited — the opposite of `latexdiff`, notorious for leaving phantom, unresolvable references behind.

### 13.2. letter-bibliography: a citation the letter makes on its own

`letter-bibliography(path, title: [References cited in this response])` a second bibliography, scoped to citations made **inside** the letter and numbered independently from the manuscript’s own.

A real project’s three-file shape (`main.typ/manuscript.typ/responses.typ`, covered in Wiring a real project below) — `manuscript.typ`:

```
#import "../../../lib.typ": *  
  
An earlier claim, supported by @jones2019.  
  
#added(<r1-1>)[A second claim, supported by @smith2020.]  
  
#bibliography("/typst-palimpsest/docs/manual-snippets/biblio-manuscript.bib")
```

An earlier claim, supported by [1].

A second claim, supported by [2].

## Bibliography

[1] A. Jones, “An outdated finding,” *Journal of Examples*, 2019.

[2] J. Smith, “A relevant finding,” *Journal of Examples*, 2020.

`responses.typ`:

```
#import "../../../lib.typ": *  
  
#reviewer(1)[  
  #exchange(<r1-1>)[Please support this claim.][  
    Done, see @smith2020. Also relevant, though not cited in the manuscript: @lee2022.  
  #pinpoint(<r1-1>)  
  ]  
]  
  
For the reviewer's convenience only:
```

```
#figure(
  table(columns: 2, [Group], [N]),
  caption: [Sample sizes, not shown in the manuscript.],
)

#letter-bibliography("/typst-palimpsest/docs/manual-snippets/biblio-letter.bib")
```

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*Please support this claim.*

Done, see [1]. Also relevant, though not cited in the manuscript: [2]. (modified on p. 1)

For the reviewer’s convenience only:

| Group | N |
|-------|---|
|-------|---|

Table R1: Sample sizes, not shown in the manuscript.

### References cited in this response

[1] J. Smith, “A relevant finding,” *Journal of Examples*, 2020.

[2] G. Lee, “A related method, not cited in the manuscript,” *Journal of Methods*, 2022.

smith2020 is [2] in the manuscript’s own bibliography but [1] in the letter’s — two independent numbering sequences, not one shared list. This is also why smith2020 had to be listed in **both** .bib files even though the manuscript already cites it: a citation made directly in the letter’s own prose only ever resolves against the .bib passed to letter-bibliography, regardless of what the manuscript’s own bibliography already knows.

**Path gotcha:** path must be root-relative (a leading /, resolved against --root), not relative to the file that calls letter-bibliography. Typst resolves a path string against the file that calls the path-consuming builtin — here, bibliography() inside letter-bibliography, in the package’s own source — not the file that wrote the string literal. A plain "responses.bib" looks next to the package’s own source and fails.

### 13.3. Letter numbering

A figure or table the letter adds **on its own** — “for the reviewer’s convenience only,” never in the manuscript, like the table above — is numbered independently from the manuscript’s, prefixed R, so it can never be confused with a manuscript one.

A figure, table, equation, or heading pinpoint(excerpt: true) re-emits **from** the manuscript is different: it keeps the manuscript’s own real number, not an R one — “Figure 1” reads “Figure 1” in the letter too, not “Figure R1”, and a quoted subsection heading reads “2.1” the same way it does in the manuscript:

```
#import "../lib.typ": *
#import "../src/letter.typ": with-letter-numbering

#document("manuscript.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #set heading(numbering: "1.1.")

  = Introduction
  = Methods

  #passage(<r1-1>)[
    #add[
```

```

    #figure(rect(width: 3cm, height: 2cm, fill: luma(220)), caption: [A subgroup analysis, added
per reviewer request.]) <fig-subgroup>
  ]
]

#passage(<r1-2>)[
  #add[
    == Sensitivity analysis <sub-sensitivity>
    Added per reviewer request.
  ]
]

#document("response.png")[
  #set page(width: 16.6cm, height: auto, margin: 12pt)
  #set text(size: 10.5pt)
  #with-letter-numbering[
    #reviewer(1)[
      #exchange(<r1-1>)[Please add a subgroup analysis.][
        Done. #pinpoint(<r1-1>, excerpt: true)
      ]
      #exchange(<r1-2>)[Please add a sensitivity analysis subsection.][
        Done. #pinpoint(<r1-2>, excerpt: true)
      ]
    ]
  ]

  For the reviewer's convenience only, not in the manuscript:

  #figure(rect(width: 3cm, height: 2cm, fill: luma(150)), caption: [A figure the letter adds on
its own.])
]
]

```

Manuscript:

## 1. Introduction

## 2. Methods

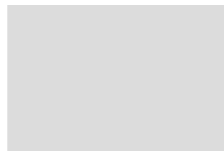


Figure 1: A subgroup analysis, added per reviewer request.

### 2.1. Sensitivity analysis

Added per reviewer request.

Response:

## Reviewer 1

### Reviewer 1 — comment 1

*Please add a subgroup analysis.*

Done. p. 1 —

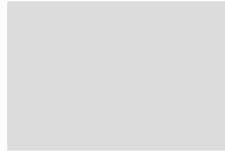


Figure 1: A subgroup analysis, added per reviewer request.

### Reviewer 1 — comment 2

*Please add a sensitivity analysis subsection.*

Done. p. 1 —

## 2.1. Sensitivity analysis

Added per reviewer request.

For the reviewer's convenience only, not in the manuscript:



Figure R2: A figure the letter adds on its own.

The letter's own figure above is captioned "Figure R2," not "Figure R1" — the two excerpts before it (the figure and the heading) still occupy the first two slots of the letter's own counters, even though what each **displays** is the manuscript's own number rather than that slot's.

This requires a label on the original — `query()` is how the letter's copy finds the manuscript's real number, and a label is what it queries by. An unlabelled figure, equation, or heading just falls back to whatever numbering already applies where the excerpt is re-emitted — the letter's own `R` sequence for a figure or table, no `R` sequence at all for an equation (equations never had one), and no number at all for a heading. This matters most in practice for headings: a figure is usually labelled anyway, for `@ref`, but a heading rarely is unless an author adds one specifically so it can be quoted this way.

## 14. Diagnostics and strict mode

Typst has no public API for a script to emit its own compiler warning, so every check below shows instead as a visible box, right at the fault — muted for anything embedded in the manuscript itself when the compile is both `variant: "clean"` and not `preview: true` (`manuscript.pdf` must never carry one, since it's the file actually sent out under review), shown regardless of mode for anything embedded in the letter (the letter is never sent out for blind review the way the manuscript is, so hiding it in one mode buys nothing). This muting logic itself lives in `@preview/contexture`, shared with every other package built on it — see below.

Two already appeared earlier, in context: a numbered anchor with no matching exchange and a bare anchor's exemption from it; `pinpoint` with no matching anchor (`on-empty:`). Four more:

```
#import "../lib.typ": *  
  
#set page(width: 16.6cm, height: auto, margin: 12pt)  
#set text(size: 10.5pt)  
#set-revisions(require-exchange: false)
```

```

*Comment with no matching passage anywhere:
#reviewer(1)[
  #exchange(<r9-1>)[A comment with nothing in the manuscript to answer.][A response anyway.]
]

*Exchange with a blank response:
#passage(<r9-2>)[#add[A real addition, matched below.]]
#reviewer(1)[
  #exchange(<r9-2>)[A real comment.][[]
]

*A mark outside any passage:
#add[Added text with no enclosing `passage`.]

*A passage with no mark and no `summary:`:
#passage(<r9-3>)[Plain text, nothing added, removed, or replaced.]

```

**Comment with no matching passage anywhere:**

### Reviewer 1

 **comment r9-1 has no matching revision in the manuscript** Reviewer 9 — comment 1

*A comment with nothing in the manuscript to answer.*

A response anyway.

**Exchange with a blank response:** [A real addition, matched below.](#) <sup>[R9-2]</sup>

### Reviewer 1

 **exchange r9-2: empty response** Reviewer 9 — comment 2

*A real comment.*

**A mark outside any passage:**  **#add outside any passage: no anchor, no page** [Added text with no enclosing passage.](#)

**A passage with no mark and no summary::**  **passage r9-3: contains no mark (did you mean touched?)**  
 Plain text, nothing added, removed, or replaced. <sup>[R9-3]</sup>

Four more exist but aren't demonstrated live here, since each needs a slightly unusual setup to trigger: two numbered exchanges sharing one anchor (duplicate exchange r1-2); xref/xcomment pointing at a label or anchor that doesn't exist anywhere (xref(<fig-x>): not found, xcomment(<r9-9>): no exchange found for this anchor); and an excerpt whose passage contains a label also referenced elsewhere in the bundle, which falls back to citing the page instead of crashing the compile outright — rare in practice, since pinpoint(excerpt: true) already strips labels from what it re-emits before this check would even trigger.

contexture.bundle(strict: true, ...) — the bundle's own strict: parameter, see below — turns every one of these into a hard compile error, in both modes, for palimpsest **and** any other package sharing the same bundle (equator's checklist diagnostics included, if one is listed too):

```
#show: contexture.bundle.with(strict: true, ...)
```

Not the default, since a manuscript mid-revision should still compile — meant for a CI gate right before submission, when every one of these should already be resolved.

## 15. Wiring a real project

Every example so far ran add/del/passage/change-list/... directly, in a single plain file, no bundle. A real project instead wires everything together explicitly, in two parts: contexture.bundle — imported from @preview/contexture, palimpsest's own dependency — is the only function anywhere in this ecosystem that ever calls

Typst's own `document(...)`; it owns the manuscript, and builds one more document per entry listed under `documents`. Palimpsest's own `letter(...)` doesn't build anything by itself — it just **describes** the response letter as one such entry. That split is what lets a second package add its own document the same way — equator's reporting-guideline checklist, say — by adding a second entry to the same list, with no risk of two competing functions each trying to call `document(...)` on their own (the closing chapter, Palimpsest in the contexture ecosystem, walks through combining the two).

A project is normally three files:

|                             |                                                                       |
|-----------------------------|-----------------------------------------------------------------------|
| <code>manuscript.typ</code> | the manuscript, annotated with <code>passage()/add()/del()/...</code> |
| <code>responses.typ</code>  | the exchanges with reviewers                                          |
| <code>main.typ</code>       | the wiring (a handful of lines)                                       |

`main.typ`:

```
#import "../..lib.typ": *
#import "../..typst-contexture/lib.typ" as contexture

#let my-template(title: none, authors: (), body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  set heading(numbering: "1.")
  align(center, text(size: 1.3em, weight: "bold")[#title])
  v(0.5em)
  body
}

#let my-letter-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  default-letter-template(body)
}

#show: contexture.bundle.with(
  template: my-template.with(title: [A Minimal Study]),
  documents: (letter(exchanges: include "shared/responses.typ", template: my-letter-template)),
)

#include "shared/manuscript.typ"
```

`manuscript.typ`:

```
#import "../..lib.typ": *

= Introduction

#added(<r1-1>)[The study rationale, expanded per the reviewer's request.]

#touched(<r1-2>)[This paragraph is unchanged; addressed in our response instead.]
```

`responses.typ`:

```
#import "../..lib.typ": *

#reviewer(1)[
  #exchange(<r1-1>)[Please expand the rationale.][
    Done. #pinpoint(<r1-1>)
  ]

  #exchange(<r1-2>)[This point seems underdeveloped.][
    We believe the current wording is sufficient. #pinpoint(<r1-2>)
  ]
]
```

Two commands, run one after the other, produce the whole project:

```
typst compile --features bundle --format bundle main.typ
typst compile --features bundle --format bundle --input variant=tracked main.typ
```

**First command** (variant defaults to "clean") — manuscript.pdf:

## A Minimal Study

### 1. Introduction

The study rationale, expanded per the reviewer's request.

This paragraph is unchanged; addressed in our response instead.

— and response.pdf:

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*Please expand the rationale.*

Done. (modified on p. 1)

#### Reviewer 1 — comment 2

*This point seems underdeveloped.*

We believe the current wording is sufficient. (see p. 1)

**Second command** (--input variant=tracked) — manuscript-tracked.pdf:

## A Minimal Study

### 1. Introduction

[The study rationale, expanded per the reviewer's request.](#) <sup>[R1-1]</sup>

This paragraph is unchanged; addressed in our response instead. <sup>[R1-2]</sup>

— and, since exchanges is set, response-tracked.pdf: the same letter, but citing and quoting **this** manuscript, the tracked one — see below for why that happens automatically, with no separate setting:

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*Please expand the rationale.*

Done. (modified on p. 1)

#### Reviewer 1 — comment 2

*This point seems underdeveloped.*

We believe the current wording is sufficient. (see p. 1)

Each compile produces exactly the file(s) for its own variant, never a mix of the two — the first never produces a tracked manuscript, and neither compile's letter is named the same as the other's.

`contexture.bundle`'s own parameters (shared by every package built on it, not specific to `palimpsest`):

`template` wraps the manuscript only — any content `-> content` function, including a real journal template used the normal way. `authors:/title:` above are this example's own stand-in template's parameters, not `bundle`'s.

`documents` an array of satellite descriptions — `letter(...)` here, possibly alongside others (see below). `()`, the default, produces the manuscript alone.

`strict bundle(strict: true, ...)` turns every diagnostic raised by **any** listed satellite, and by `palimpsest`'s own marks/exchanges, into a hard compile error — see Diagnostics and strict mode above.

`letter(...)`'s own parameters, `palimpsest`-specific:

`exchanges` the already-evaluated content of the responses file — `include "responses.typ"`, as shown above. Also decides whether a letter is produced at all — see below.

`template` wraps the letter only, separately from the manuscript's own `template:` above — `auto` (the default) uses `default-letter-template`, a title and nothing else. The two templates never mix: a figure/table/heading style set inside one has no effect on the other.

Each source file needs its own `#import "@preview/palimpsest:0.1.0": * — #include` doesn't share scope, so `manuscript.typ` and `responses.typ` both import it too, even though only `main.typ` calls `letter/bundle`. Only `main.typ` needs `contexture` itself.

### 15.1. The letter, automatically — matched to the manuscript you actually send

Whether a compile produces a response document isn't something you configure — there's nothing sensible for it to mean independently of `exchanges`: writing responses with no letter to hold them, or asking for a letter with nothing written, are both non-cases. So it's derived: `letter(...)`'s own applicable rule builds a letter whenever `exchanges` isn't none, matching **this compile's own** variant — `response.pdf` from the clean compile, `response-tracked.pdf` from the tracked one, each citing and quoting its own manuscript. This changes nothing about the command line — it's still the exact same two commands shown above.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

#let my-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  body
}

#let my-letter-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
```

```

    default-letter-template(body)
  }

  #let exchanges = reviewer(1)[
    #exchange(<r1-1>)[This claim needs more support.][
      We have expanded this sentence. #pinpoint(<r1-1>, excerpt: true)
    ]
  ]

  #show: contexture.bundle.with(
    template: my-template,
    documents: (letter(exchanges: exchanges, template: my-letter-template)),
  )

  #passage(<r1-1>)[The treatment #rep[has an effect][has a clinically meaningful effect] on
  survival.]

```

Compiled clean, response.pdf quotes the accepted wording only:

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*This claim needs more support.*

We have expanded this sentence. **p. 1** — The treatment has a clinically meaningful effect on survival.

Compiled with `--input variant=tracked`, the same exchanges also produces `response-tracked.pdf` — a name distinct from `response.pdf`, so neither compile can silently overwrite the other’s output. Its excerpt shows the struck-through old wording next to the underlined new wording, because `pinpoint(excerpt: true)` with no explicit `mode`: always follows whichever mode the **current** compile is running under (see `pinpoint` above):

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*This claim needs more support.*

We have expanded this sentence. **p. 1** — The treatment ~~has an effect~~ has a clinically meaningful effect on survival.

The one thing that **is** a command-line choice, not a fixed property of the project, is skipping the letter for a single run even though `exchanges` is set — a fast, manuscript-only preview while drafting, without touching `main.typ`. This is `contexture.bundle`’s own mechanism, `--input only=`, not something specific to the letter — listing nothing after the `=` means “the manuscript alone, whatever’s in `documents:`”:

```
typst compile --features bundle --format bundle --input only= main.typ
```

Naming a satellite explicitly (`--input only=response`) restricts the compile to just that one, still subject to its own rules (a satellite gated on something else, like `equator`’s checklist below, doesn’t get forced on just because it’s named here).

What each command writes, with `exchanges` set:

| Command | default | <code>--input only=</code> |
|---------|---------|----------------------------|
|---------|---------|----------------------------|

|               |                                                |                        |
|---------------|------------------------------------------------|------------------------|
| 1st (clean)   | manuscript.pdf<br>response.pdf                 | manuscript.pdf         |
| 2nd (tracked) | manuscript-tracked.pdf<br>response-tracked.pdf | manuscript-tracked.pdf |

## 16. Palimpsest in the contexture ecosystem

Palimpsest is one of two packages built on `contexture`, a small shared package neither of them ships duplicated logic for. This chapter explains what `contexture` actually contributes, introduces the other package built on it, and covers what changes when both are used together.

### 16.1. What `contexture` does

Everything in this manual that looks up a **real** page number across two documents — `response.pdf` citing exactly where in `manuscript.pdf` a change landed — relies on Typst’s experimental `bundle export`, which lets one compile produce several documents that can query each other’s final layout. `contexture` is the small toolkit that turns that raw capability into something a package author can build on without reinventing it each time:

- an **anchor** primitive — mark a spot in one document, read it back from any other, by its real page; `pinpoint` and `xref` are both built directly on it;
- a **shared compile engine** (`bundle`) — the single point that ever calls Typst’s own `document(...)`, so palimpsest’s letter and, say, another package’s own generated document can both be listed side by side without competing to own the compile;
- two independent **compile flags**, `variant` and `preview` — palimpsest reads `variant` for its own clean/tracked distinction (`--input variant=tracked` throughout this manual); `preview` is a second, independent axis a package can use for its own purposes (`equator` uses it for its `check()` highlighting, below);
- a shared **diagnostics** mechanism and `strict` flag — what every warning box and `strict: true` in this manual are actually built from.

`passage()/add()/del()/rep()` are built directly on `contexture`’s anchor primitive; `letter(...)` is a thin description handed to its shared compile engine. None of this needs to be learned to use palimpsest as documented above — it’s mentioned here because the same foundation is shared with the package below, which is what makes combining the two straightforward rather than a rewrite.

### 16.2. `@preview/equator`: reporting-guideline checklists

`equator` is a sibling package for a different problem: filling in a reporting-guideline grid (CONSORT, PRISMA, SPIRIT, STARD, STROBE...) automatically, by marking where each item is answered in the manuscript (`check(id, body)`) and letting the grid cite the real page each one landed on. See `@preview/equator`’s own manual for the full picture; nothing in it is needed to use palimpsest on its own.

### 16.3. Combining the two

Because `contexture.bundle` — not palimpsest — owns `documents:`, adding a second package’s own document alongside `letter(...)` is just a second entry in the same array, not a second engine to reconcile. Here, a CONSORT reporting checklist (`@preview/equator`) is produced from the **same** manuscript, in the **same** compile, as the reviewer response letter:

```
#import "../lib.typ": *
#import "../typst-equator/lib.typ": *
#import "../typst-contexture/lib.typ": bundle

// A tiny made-up grid, not a real reporting guideline — kept small so the
// generated checklist fits on one page for this illustration; a real
// project would pass e.g. `checklists.consort` here instead.
#let tiny-checklist = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Randomisation", group: none, id: "1",
     description: [How the allocation sequence was generated.]),
    (section: "Methods", topic: "Outcome assessment", group: none, id: "2",
     description: [How the primary outcome was assessed.]),
```

```

    ),
  )

#let my-template(body) = {
  set page(width: 16.6cm, height: auto, margin: 12pt)
  set text(size: 10.5pt)
  body
}

#let exchanges = reviewer(1)[
  #exchange(<r1-1>)[Please clarify whether outcome assessment was blinded.][
    Blinding is now specified. #pinpoint(<r1-1>)
  ]
]

#show: bundle.with(
  template: my-template,
  documents: (
    letter(exchanges: exchanges),
    checklist(checklist: tiny-checklist),
  ),
)

// Item 1 has nothing to do with this reviewer exchange – check() just
// renders its own text, independently, at its own spot. No overlap, no
// duplication to worry about: this is the common case.
#check("1")[Randomisation used a computer-generated sequence.]

// Item 2, though, IS the reviewer's exchange: the added text is both a
// tracked revision AND the manuscript's answer to item 2. check(id, body)
// and passage() would each render body – used together on the exact
// same span, that prints it twice. check(id)'s bare, point-marker form
// registers item 2's coverage without rendering anything; passage() is
// the only call that actually prints the text, once.
#passage(<r1-1>)[
  The primary outcome was assessed #add[by a rater blinded to group assignment].
]
#check("2")

```

manuscript.pdf — item 1 is unrelated to the reviewer exchange, so `equator.check(...)` just renders its own text at its own spot, independently of `passage(...)`; item 2's text, though, genuinely **is** the reviewer exchange, so it's rendered exactly once, by `passage(...)` alone (see below for why):

Randomisation used a computer-generated sequence.

The primary outcome was assessed by a rater blinded to group assignment.

response.pdf, citing the manuscript's real page as always:

## Response to Reviewers

### Reviewer 1

#### Reviewer 1 — comment 1

*Please clarify whether outcome assessment was blinded.*

Blinding is now specified. (modified on p. 1)

### Tiny reporting checklist

| Section / Topic    | No | Description                                | Reported on page no. |
|--------------------|----|--------------------------------------------|----------------------|
| <b>Methods</b>     |    |                                            |                      |
| Randomisation      | 1  | How the allocation sequence was generated. | 1                    |
| Outcome assessment | 2  | How the primary outcome was assessed.      | 1                    |

Three documents, one manuscript, one compile — and `--input variant=tracked/--input preview=true` (equator's own preview flag, for its `check()` anchors) both still work exactly as shown throughout this manual, independently of whichever other packages are listed in `documents:`.

**Never nest one package’s marking function inside another’s** — `#check(...)[#passage(...)[...]]` and the reverse each break something, for the same underlying reason both times: `passage()`’s own visual rendering, and `check()`’s own preview-mode highlighting, are each wrapped in a `context` block (needed to read live style state) — and a `context` block is structurally opaque to anything trying to inspect its contents **before** layout, the same limitation this manual already documents for `pinpoint`’s label handling. Nest `passage()` inside `check()` and `check()`’s own blank-content self-check can no longer see the real text inside — it misreports the passage as empty, in **any** mode, not just under `preview: true`. Nest `check()` inside `passage()` and, under `preview: true` specifically, `passage()`’s own scan for `add/del/rep` marks can no longer see them — it misreports “contains no mark”. Two different symptoms, one cause, and no nesting order avoids it.

**But don’t just call them as two independent, side-by-side renders on the exact same span either** — both `check(id, body)` and `passage()` render their own body; two calls on identical text print it twice, as plain, visible duplication (item 1 above is fine precisely because it’s a **different** span from anything palimpsest touches — the common case, and the one worth defaulting to whenever a checklist item and a reviewer exchange simply don’t coincide). When they do coincide — the reviewer’s requested change **is** the manuscript’s answer to a checklist item, item 2 above — use the bare `check(id)` (no second argument) instead of `check(id, body)`: same metadata, same page resolution in `checklist.pdf`, but it renders nothing but a small superscripted `id` under `--input preview=true` — nothing that duplicates or nests. `passage(...)` stays the one call that actually prints the text and handles its tracked-mode marks; `check(id)` just tells equator where to find it.

Not a palimpsest-specific rule: it applies to any two `contexture`-based packages whose marking functions both render content over the same span, present or future — the fix is the same shape every time, one call renders, any other call that needs to know about that same span registers via its own package’s bare, non-rendering form instead of its normal marking call (equator’s own `check` folds both shapes into one function, the same way palimpsest’s own `passage(anchors, body)/passage(body)` already does).

## 16.4. Where to go next

- The mechanics behind all of this, on their own, with no notion of revisions or checklists attached: `@preview/contexture`’s manual.
- Reporting-guideline checklists that cite the real pages: `@preview/equator`’s manual.
- Everything about manuscript revisions and reviewer letters on their own: the rest of this manual, from Your first revision round onward.